

PROMPT TO LOCK VOCABULARY

Use this once at start of every module:

CANONICAL VOCABULARY (STRICT):

You must use ONLY the following backend terms:

Core:

tenant, branch, user, role

CRM:

lead, pipeline, stage, activity

People:

member, profile, contact

Engagement:

offering, session, enrollment, booking

Commerce:

plan, subscription, invoice, payment

Operations:

attendance, event, task

Lifecycle:

lifecycle, status, metric

System:

config, audit_log, pii_data

RULES:

- Do NOT introduce new terminology without justification
- Do NOT use industry-specific words (student, course, resident, etc.)
- Do NOT rename these entities
- All models, APIs, and fields must follow this vocabulary

VALIDATION:

Before finalizing, confirm:

- all names are from canonical vocabulary
- no industry-specific terms are used

Check:

- any violation of canonical vocabulary?
- any hardcoded UI terminology?

Design Terminology API

TASK: Implement Terminology Mapping Module APIs

CONTEXT:

We have a canonical vocabulary system where backend uses generic keys (e.g., "member", "session", "plan") and UI maps them to tenant-specific labels.

REQUIREMENTS:

1. MODEL:

Create a Terminology model:

- tenant (FK)
- key (string, required)
- label (string, required)
- unique constraint: (tenant, key)

2. APIs:

A. GET /api/terminology/

- Return all key-label mappings for the current tenant
- Response format:

```
{
  "member": "Student",
  "session": "Class"
}
```

B. PUT /api/terminology/bulk/

- Accept dictionary of key-label pairs
- Update or create entries
- Must be atomic (transaction)

C. POST /api/terminology/

- **Update single key**

3. RULES:

- **MUST enforce tenant isolation**
- **MUST use request.tenant (multi-tenant system)**
- **MUST enforce RBAC:**
 - **Only admin/owner can update**
- **MUST NOT allow duplicate keys per tenant**
- **MUST NOT hardcode any terminology**
- **MUST use service layer (no business logic in views)**

4. OUTPUT:

Provide:

- **Django model**
- **Service layer functions**
- **API views (DRF or Django views)**
- **Serializer (if needed)**
- **Validation rules**

5. VALIDATION:

Before finalizing:

- **Check tenant isolation**
- **Check RBAC enforcement**
- **Check duplicate key handling**
- **Check performance (no unnecessary queries)**